



Développement WEB: JavaScript Avancée

DUT Génie Informatique

Pr. Tarek AIT BAHA

t.aitbaha@uiz.ac.ma

Année universitaire : 2025/2026

Objectifs du cours

- Comprendre les bases de la programmation JavaScript,
- Variables, types de données, opérateurs,
- Structures de contrôle (boucles, conditions),
- Fonctions et portée des variables,
- Utiliser les outils de développement JavaScript,
- Manipuler le DOM et réagir aux événements,
- Utiliser des fonctionnalités modernes (ES6+),
- Comprendre la programmation asynchrone,

Objectifs d'apprentissage

À la fin de ce chapitre, vous serez capable de:

- Expliquer le rôle de JavaScript dans le développement web,
- Configurer un environnement de développement JavaScript,
- Distinguer l'exécution JS dans le navigateur vs Node.js,
- Manipuler le DOM et le BOM de façon basique.

Plan du cours

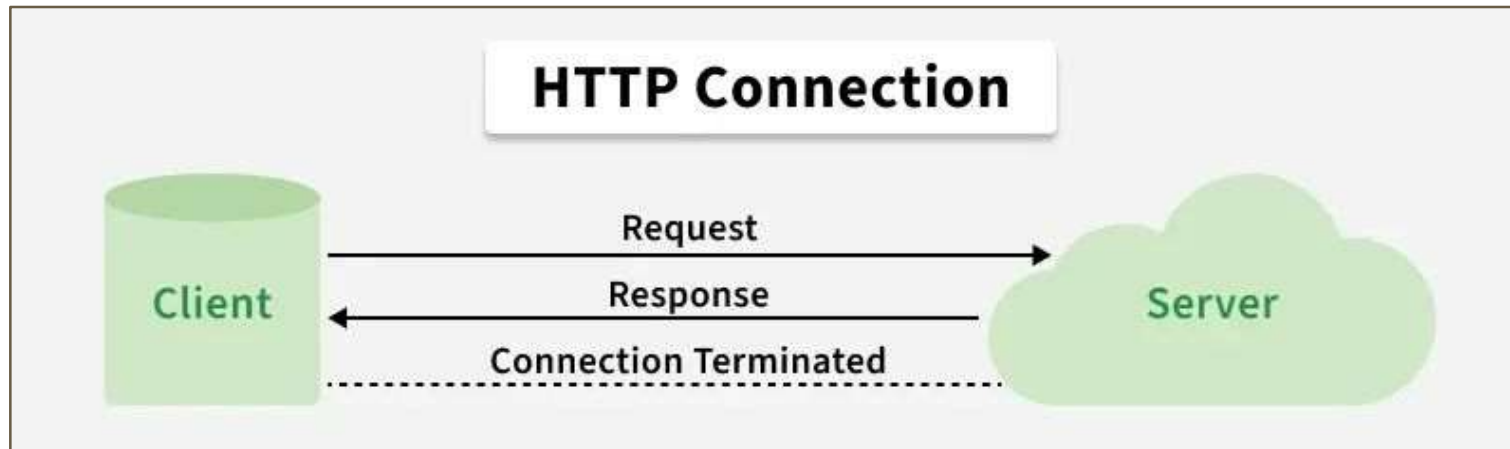
- Semaine 1 – Concepts de base et applications
- Semaine 2 – Variables et Types de données
- Semaine 3 – Fonctions
- Semaine 4 – Gestion des Événements
- Semaine 5 – Programmation Asynchrone
- **Semaine 6 – Requêtes HTTP en JavaScript**

Semaine 6:

Requêtes HTTP en JavaScript

Introduction aux Requêtes HTTP

- **HTTP (HyperText Transfer Protocol)** est la base de la communication web.
- Modèle client-serveur : les clients (navigateurs) envoient des requêtes, les serveurs répondent.
- JavaScript permet d'effectuer des requêtes HTTP dynamiquement sans rechargement de page.
- Approche moderne : API `fetch` avec la syntaxe `async/await`



Programme de la séance

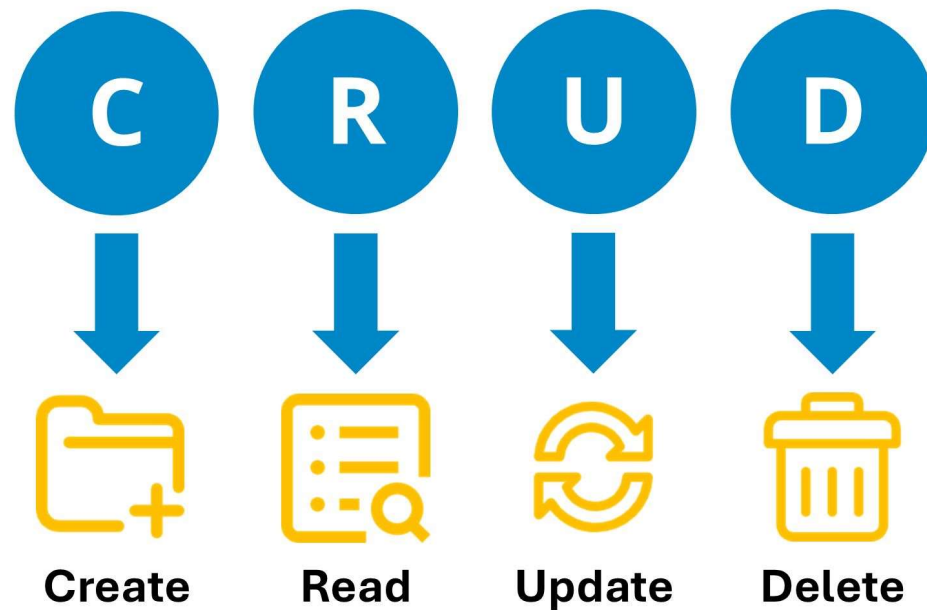
1. Bases des requêtes HTTP (GET , POST , PUT , DELETE)
2. Utilisation de l'API Fetch
3. Gestion des réponses et des erreurs
4. Manipulation de données JSON
5. Configuration d'en-têtes personnalisés
6. Exercices pratiques

Méthodes HTTP Principales

- **GET** : Récupère des données (lecture)
- **POST** : Crée de nouvelles données
- **PUT** : Met à jour entièrement une ressource existante
- **DELETE** : Supprime une ressource
- **PATCH** : Met à jour partiellement une ressource (non couvert aujourd'hui)

API REST

- **REST** (Representational State Transfer) est un style d'architecture pour les APIs
- Utilise les méthodes HTTP pour effectuer des opérations CRUD



API REST

- **REST** (Representational State Transfer) est un style d'architecture pour les APIs
- Utilise les méthodes HTTP pour effectuer des opérations CRUD
- Les ressources sont identifiées par des URL
- Les données sont généralement échangées au format JSON
- Exemple d'URL REST : <https://api.exemple.com/posts/1>
 - **posts** : ressource
 - **1** : identifiant de la ressource

End point d'une API REST

- Un **end point** est une URL spécifique d'une API où une ressource peut être accédée
- Exemple : <https://jsonplaceholder.typicode.com/posts/1>
 - **posts** : ressource (collection de posts)
 - **/1** : identifiant du post spécifique
- Les **end points** peuvent accepter différentes méthodes HTTP pour effectuer des opérations.
- Exemple de requêtes sur un end point :
 - **GET** **/posts** : Récupérer tous les posts
 - **POST** **/posts** : Créer un nouveau post
 - **PUT** **/posts/1** : Mettre à jour le post avec ID

Autres types API

- **SOAP** : Protocole basé sur XML, plus complexe
- **GraphQL** : Langage de requête pour les APIs, plus flexible
- **gRPC** : Protocole RPC basé sur HTTP/2, utilisé pour les microservices
- **WebSocket** : Communication bidirectionnelle en temps réel
- **MQTT** : Protocole léger pour les appareils IoT

JSON (JavaScript Object Notation)

- Format léger d'échange de données.
- Facile à lire et à écrire pour les humains.
- Facile à analyser et à générer pour les machines.
- Utilisé pour représenter des objets JavaScript.
- Exemple de JSON :

```
1 {  
2   "userId": 1,  
3   "id": 1,  
4   "title": "Titre du post",  
5   "body": "Contenu du post"  
6 }
```

- Peut être converti en objet JavaScript avec `JSON.parse()`
- Peut être converti en chaîne JSON avec `JSON.stringify()`

Fetch

- Interface moderne pour effectuer des requêtes HTTP en JavaScript.
- Remplace `XMLHttpRequest` (plus ancien et plus verbeux).
- Utilise les **Promesses (Promise)** pour une gestion asynchrone plus élégante.
- Compatible avec `async/await` pour un code plus lisible.

```
1 // Structure de base d'une requête fetch
2 fetch(url, options)
3   .then(response => response.json())
4   .then(data => console.log(data))
5   .catch(error => console.error(error));
```

Syntaxe async/await

- Facilite l'écriture de code asynchrone de manière synchrone
- Améliore la lisibilité et la gestion d'erreurs

```
1  async function getData() {  
2    try {  
3      const response = await fetch(url);  
4      const data = await response.json();  
5      console.log(data);  
6    } catch (error) {  
7      console.error(error);  
8    }  
9  }
```

Demonstrations

- Nous utiliserons [JSONPlaceholder](#)
 - API de test gratuite pour s'exercer aux requêtes REST
 - Fournit des données fictives pour les posts, utilisateurs, commentaires, etc.
- Nous allons effectuer des opérations CRUD sur les posts
- Nous allons utiliser `async/await` pour simplifier le code

Requête **GET** - Récupération de données

```
1  async function getPosts() {
2    try {
3      // Envoi de la requête GET
4      const response = await fetch('https://jsonplaceholder.typicode.com/posts');
5
6      // Conversion de la réponse en JSON
7      const data = await response.json();
8
9      // Affichage dans la console
10     console.log('Liste des posts :', data);
11   } catch (error) {
12     console.error('Erreur lors de la récupération des posts :', error);
13   }
14 }
```

Affichage dans la Page

```
<div class="container">
  <button onclick="displayPosts()">Afficher les Posts</button>
  <div id="postsContainer" class="posts-list"></div>
</div>

<script>
async function displayPosts() {
  try {
    const response = await fetch('https://jsonplaceholder.typicode.com/posts');
    const posts = await response.json();
    const container = document.getElementById('postsContainer');

    container.innerHTML = ''; // Vider le conteneur

    // Afficher les 10 premiers posts
    posts.slice(0, 10).forEach(post => {
      const postElement = document.createElement('div');
      postElement.className = 'post';

      postElement.innerHTML = `
        <h3>${post.title}</h3>
        <p>${post.body}</p>
      `;

      container.appendChild(postElement);
    });

  } catch (error) {
    console.error('Erreur lors de la récupération des posts :', error);
  }
}
</script>
```

GET avec Paramètres d'URL

- Vous pouvez ajouter des paramètres à l'URL pour filtrer les résultats
- Exemple : <https://jsonplaceholder.typicode.com/posts?userId=1> pour récupérer les posts de l'utilisateur **1**
- Vous pouvez également utiliser d'autres paramètres pour affiner votre recherche, comme `?_limit=5` pour limiter le nombre de résultats.
- Vous pouvez également utiliser `?_sort=title` pour trier les résultats par titre.
- Exemple de code pour récupérer les posts d'un utilisateur spécifique :

GET avec Paramètres d'URL

- Exemple de code pour récupérer les posts d'un utilisateur spécifique :

```
<script>
async function getUserPosts(userId) {
  try {
    const url = `https://jsonplaceholder.typicode.com/posts?userId=${userId}&_limit=5&_sort=title`;

    const response = await fetch(url);
    const posts = await response.json();

    const container = document.getElementById('postsContainer');
    container.innerHTML = ''; // Vider le conteneur

    posts.forEach(post => {
      const postElement = document.createElement('div');
      postElement.className = 'post';

      postElement.innerHTML = `
        <h3>${post.title}</h3>
        <p>${post.body}</p>
      `;

      container.appendChild(postElement);
    });
  } catch (error) {
    console.error('Erreur :', error);
  }
}
</script>
```

Syntaxe fetch pour POST

- Préciser la methode **POST** dans l'objet d'options.
- Ajouter les en-têtes **Content-Type** pour indiquer le format des données envoyées.
- Préciser le corps de la requête avec **body** (doit être une chaîne JSON).

Exemple de code pour création des données avec POST

```
<script>
async function createPost() {
  // Définition de l'objet à poster
  const newPost = {
    title: 'Mon nouveau post',
    body: 'Contenu du post...',
    userId: 42
  };

  try {
    const response = await fetch('https://jsonplaceholder.typicode.com/posts', {
      method: 'POST', // Méthode HTTP
      headers: {
        'Content-Type': 'application/json' // Format des données
      },
      body: JSON.stringify(newPost) // Conversion en JSON
    });
```

```
    const data = await response.json();

    console.log('Post créé :', data);

    // Optionnel : affichage dans le DOM
    const container = document.getElementById('postsContainer');
    const postElement = document.createElement('div');
    postElement.className = 'post';

    postElement.innerHTML = `
      <h3>${data.title}</h3>
      <p>${data.body}</p>
      <small>User ID: ${data.userId}</small>
    `;

    container.prepend(postElement);

  } catch (error) {
    console.error('Erreur lors de la création du post :', error);
  }
}
</script>
```

Headers dans fetch

- Les **headers** sont des métadonnées envoyées avec la requête HTTP.
- Ils fournissent des informations sur le type de contenu, l'authentification, etc.

Types de Headers courants

- **Content-Type**: Indique le type de contenu envoyé (ex: application/json)
- **Authorization**: Utilisé pour l'authentification (ex: Bearer TOKEN)
- **Accept**: Indique le type de contenu attendu en réponse (ex: application/json)
- **User-Agent**: Informations sur le client (navigateur, version...)

Type de contenu courants

- `application/json`: Utilisé pour les APIs REST , envoi de données JSON
- `text/html`: Utilisé pour du HTML brut , utilisé pour les pages web
- `text/plain`: Texte brut, utilisé pour des données textuelles simples

Type authorisation courants

- **Bearer**: Utilisé pour les tokens d'authentification (ex: JWT)
- **OAuth**: Protocole d'autorisation pour accéder à des ressources protégées
- **Basic**: Authentification de base (nom d'utilisateur et mot de passe)

Exemple authorisation Bearer token dans les headers

```
1 async function fetchWithAuth() {  
2   const response = await fetch('https://api.exemple.com/ressource', {  
3     method: 'GET',  
4     headers: {  
5       'Content-Type': 'application/json',  
6       'Authorization': 'Bearer YOUR_TOKEN_HERE',  
7     }  
8   });  
9   const data = await response.json();  
10  console.log('Données récupérées :', data);  
11 }  
12
```

Syntaxe PUT

- Utilisé pour mettre à jour une ressource existante
- Envoie l'objet complet à mettre à jour
- Remplace la ressource existante par la nouvelle version

```
async function updateResource() {  
  try {  
    const response = await fetch('URL_DE_LA_RESSOURCE', {  
      method: 'PUT',  
      headers: {  
        'Content-Type': 'application/json'  
      },  
      body: JSON.stringify(data)  
    });  
  
    const result = await response.json();  
    console.log('Succès :', result);  
  
  } catch (error) {  
    console.error('Erreur :', error);  
  }  
}
```

Exemple de code pour mettre à jour une ressource avec PUT

```
async function updatePost() {
  const updatedPost = {
    id: 1,
    title: 'Titre mis à jour',
    body: 'Contenu mis à jour',
    userId: 1
  };

  try {
    const response = await fetch('https://jsonplaceholder.typicode.com/posts/1', {
      method: 'PUT', // Méthode HTTP
      headers: {
        'Content-Type': 'application/json'
      },
      body: JSON.stringify(updatedPost)
    });
  }
```

```
    const data = await response.json();

    console.log('Post mis à jour :', data);

    // Optionnel : affichage dans le DOM
    const container = document.getElementById('postsContainer');
    const postElement = document.createElement('div');
    postElement.className = 'post';

    postElement.innerHTML = `
      <h3>${data.title}</h3>
      <p>${data.body}</p>
      <small>User ID: ${data.userId}</small>
    `;

    container.prepend(postElement);

  } catch (error) {
    console.error('Erreur lors de la mise à jour :', error);
  }
}
```

Requête **PATCH** - Mise à jour partielle

- Utilisé pour mettre à jour une partie d'une ressource existante.
- Envoie uniquement les champs à mettre à jour.
- Exemple : Mettre à jour uniquement le titre d'un post.
- Syntaxe similaire à **PUT** , mais avec **PATCH** comme méthode.

Exemple de code pour mettre à jour une ressource avec PATCH

```
async function patchPost() {
  const updatedFields = {
    title: 'Titre mis à jour partiellement'
  };

  try {
    const response = await fetch('https://jsonplaceholder.typicode.com/posts/1', {
      method: 'PATCH',
      headers: {
        'Content-Type': 'application/json'
      },
      body: JSON.stringify(updatedFields)
    });
  }
```

```
    const data = await response.json();
    console.log('Post mis à jour partiellement :', data);

    // Optionnel : affichage dans le DOM
    const container = document.getElementById('postsContainer');
    const postElement = document.createElement('div');
    postElement.className = 'post';

    postElement.innerHTML = `
      <h3>${data.title}</h3>
      <p>${data.body || '(contenu inchangé)'}</p>
      <small>User ID: ${data.userId}</small>
    `;

    container.prepend(postElement);

  } catch (error) {
    console.error('Erreur PATCH :', error);
  }
}
```

Requête **DELETE** - Suppression de données

- Utilisé pour supprimer une ressource existante.
- Envoie une requête avec la méthode **DELETE**.
- Exemple : Supprimer un post spécifique.
- Syntaxe similaire aux autres requêtes, mais avec **DELETE** comme méthode.
- Pas besoin d'envoyer de corps (body) dans la requête.

Exemple de code pour supprimer un poste

```
async function deletePost() {
  try {
    const response = await fetch('https://jsonplaceholder.typicode.com/posts/1', {
      method: 'DELETE'
    });

    if (response.ok) {
      console.log('Post supprimé.');
```

```
      document.getElementById('output').textContent = 'Post #1 supprimé.';
    } else {
      console.error('Échec de la suppression, code :', response.status);
      document.getElementById('output').textContent = 'Erreur lors de la suppression.';
    }

  } catch (error) {
    console.error('Erreur DELETE :', error);
  }
}
```